TECHNICAL WHITE PAPER

Shokunin 職人
The AI Engineering Ecosystem

38 specialized skills, ChromaDB persistent memory, 3 MCP servers, 4 subagents. Cross-platform Windows and Linux. Six AI coding runtimes supported. Zero cost, open source, MIT license.

Elias Oulkadi
Version 4.2 · May 2026
github.com/EliasOulkadi/shokunin

Contents

EXECUTIVE SUMMARY

Executive Summary

Shokunin (職人, Japanese for artisan) is an open source ecosystem that transforms AI coding agents into production-grade engineering workstations. 38 domain-specific skills, persistent ChromaDB memory across sessions, and cross-platform support eliminate the blank-slate problem that affects every AI coding session.

When an engineer opens a new AI coding session, the agent has zero context of the project, past decisions, or codebase. Shokunin solves this through a three-layer memory system that preserves context across sessions, a library of 38 specialized skills that teach the agent domain-specific workflows, and a set of platform-native scripts that automate maintenance and session management.

The ecosystem is fully open source (MIT license), runs entirely on local hardware, and supports six AI coding runtimes across Windows and Linux. It requires no cloud services, no subscriptions, and no telemetry.

Key Metrics

CATEGORY	COUNT	DETAILS
Skills	38	Infrastructure, backend, frontend, mobile, quality, content, productivity, documents, system
Scripts	20	8 PowerShell (.ps1), 6 bash (.sh), 3 Python (.py), 3 CI workflows
Platforms	2	Windows (PowerShell 5.1+) and Linux (bash 4+)
Runtimes	6	OpenCode, Claude Code, Cline, Cursor, Continue.dev, Windsurf
Commits	52	Active development since Q4 2025
Memory entries	11+	ChromaDB vector database with session tracking
Bug fixes	10	Critical bugs identified and resolved during audit
CI pipelines	3	Validation, memory tests, release automation
Documentation PDFs	7	Comprehensive guides, changelogs, quick reference
Installers	2	Windows (467 lines) and Linux (206 lines), both idempotent

> One command install:
> Windows: `irm https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.ps1 | iex`
> Linux: `bash <(curl -sL https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.sh)`

01 · PROBLEM

The Problem: AI Context Loss

Every AI coding session starts with a blank slate. The agent has no memory of the project, the codebase, or the decisions made in previous sessions. This is the single biggest productivity drain in AI-assisted development.

The Blank-Slate Problem

When an engineer opens an AI coding agent, the conversation begins with zero context. The agent does not know what project the user is working on, what decisions were made previously, what the codebase structure looks like, or what conventions the team follows. Every session starts from scratch.

In practice, this means:

- Engineers repeat context in every session: "We are building a React app with TypeScript, using Prisma for the database, deployed on AWS ECS."
- Decisions made in one session are lost in the next: the agent may recommend a different approach than what was agreed upon previously.
- Domain-specific knowledge (Dockerfile patterns, Kubernetes manifests, OAuth flows) must be re-taught or re-prompted.
- Long-running projects accumulate tribal knowledge that never transfers to the AI agent.

The Skills Gap

Most AI agents operate on general knowledge. They know about Docker, Kubernetes, and authentication in theory, but they do not have structured, production-tested workflows for these domains. When asked to "containerize this app" or "add OAuth login," the agent improvises based on training data rather than executing a validated procedure.

This leads to security misconfigurations, non-optimal patterns, and inconsistent results across sessions.

The Automation Gap

Beyond conversation, there is no automated context capture. Sessions end and the context is lost. Terminal output disappears. File changes are not tracked. There is no mechanism to persist "what happened" across session boundaries.

02 · SOLUTION

Solution Overview

Shokunin addresses these three gaps through a cohesive ecosystem of skills, memory, and automation. The approach combines structured domain knowledge with persistent vector storage and platform-native scripting.

Core Principles

PRINCIPLE	DESCRIPTION
Local-first	All data stays on the user's machine. No cloud, no telemetry, no subscriptions. ChromaDB runs locally as an embedded database.
Zero cost	MIT license, NVIDIA free API tier, Ollama for local inference. No paid tiers, no lock-in.
Auto-activating	Skills activate by matching task descriptions. No manual skill selection. The agent reads skill descriptions at startup and chooses the right one.
Persistent memory	Three capture layers ensure no context is lost: agent-driven saves, periodic checkpoints, and terminal output capture on exit. ChromaDB provides semantic search across all past entries.
Cross-platform	Shared Python core with platform-specific wrappers. Windows and Linux share the same skills, memory, and MCP servers. Only the installer and scheduler differ.
Multi-runtime	Not tied to OpenCode. Skills, memory, and scripts work across six different AI coding runtimes. Template configurations are provided for each.
Idempotent install	Installers create backups before modifying any existing file. Safe to re-run. No destructive operations.

How It Works

When the user types `opencode`, a profile function shadows the direct command and routes it through a wrapper script. The wrapper sets environment variables with session metadata, writes a session identifier to disk, pings the MCP servers for availability, and launches the real opencode binary. During the session, the agent saves checkpoints, decisions, and file changes to ChromaDB via chroma-helper.py. A background timer saves additional checkpoints every five minutes. When the session ends, the wrapper reads the terminal output buffer, strips ANSI codes, and saves the content to both ChromaDB and markdown files.

On the next session start, the agent searches ChromaDB for relevant past entries. If the semantic search returns results, the agent presents them to the user. If no relevant results are found, the most recent entries are shown as context.

> Key differentiator: Shokunin's memory system does not depend on the MCP server. The agent saves via `chroma-helper.py` directly, which writes to ChromaDB through the Python API. This means memory works regardless of MCP server availability or runtime type.

03 · ARCHITECTURE

Architecture

Five independent layers that each serve a specific role. Every layer can be replaced, extended, or run on a different operating system without affecting the others.

Layer Architecture

LAYER	COMPONENTS	RESPONSIBILITIES
1. INTERFACE	OpenCode, Claude Code, Cline, Cursor, Continue.dev, Windsurf	The AI coding runtime the user interacts with. Shokunin adapts to each through platform-specific configuration files.
2. AGENT	38 SKILL.md files, superpowers plugin, 4 subagents	Domain expertise through auto-activating skills. Parallel code review and debugging via subagents.
3. INFRASTRUCTURE	3 MCP servers (filesystem, fetch, memory), ChromaDB	Direct system access and persistent vector storage. MCP servers start on demand.
4. AI PROVIDERS	NVIDIA Kimi K2.6, OpenAI, Anthropic, Ollama (local)	Provider-agnostic model access. Change the model in opencode.json, everything else works the same.
5. SYSTEM	PowerShell/bash scripts, Task Scheduler/crontab	Platform-specific automation. Only layer that varies between Windows and Linux.

Design Decisions

- Layer isolation: Each layer can change independently. Skills do not depend on specific MCP servers. Memory does not depend on specific runtimes. Scripts do not depend on specific AI providers.
- Core cross-platform: chroma-helper.py, mcp-server.py, and all 38 SKILL.md files are identical on Windows and Linux. Path resolution detects the OS automatically.
- Fallback-first: Every critical operation has at least two fallback paths before reporting failure. Memory saves retry twice, buffer capture tries three strategies, search falls back from semantic to recent entries.
- Additive changes only: The entire ecosystem follows a strict policy of never modifying existing content. All improvements are additions: new scripts, new sections, new templates.

The Skills System

Each skill is a SKILL.md file that teaches the agent how to handle a specific domain. Skills include procedural workflows, error handling, anti-patterns, and cited sources. They auto-activate based on the task description.

Skill Format

Every skill follows a strict structure validated by CI:

- YAML frontmatter: name, trigger-optimized description, license, compatibility, metadata, allowed tools
- Workflow: Numbered steps with decision tables and code examples
- Error handling: Scenario/cause/fix table for common issues
- Production checklist: Verifiable items before marking a task done
- Anti-patterns: What not to do, with concrete fixes
- Cited sources: Official documentation, OWASP, NIST, RFCs

Skills by Domain

Infrastructure (6)

SKILL	TRIGGER	DEPTH
docker	"Containerize my app"	Multi-stage builds, BuildKit cache, vulnerability scanning, compose
kubernetes	"Deploy to K8s"	Gateway API, service mesh, HPA, PDB, pod debugging
terraform	"Set up AWS infra"	Stacks, test framework, pre/postconditions, state surgery
ci-cd	"Add CI pipeline"	GitHub Actions, GitLab CI, CircleCI, OIDC auth, canary
db-admin	"Backup database"	PostgreSQL backup, replication, WAL archiving, PITR
runbook-gen	"Site is down"	Incident runbooks, post-mortems, war room setup

Backend (4)

SKILL	TRIGGER	DEPTH
auth-architect	"Add login"	OAuth2+PKCE, WebAuthn, JWT rotation, RBAC, OWASP Top 10
api-forge	"Design an API"	REST/GraphQL, OpenAPI 3.1, webhooks, idempotency, cursor pagination
db-sculptor	"Optimize query"	B-tree/GIN/GiST/BRIN/Hash indexes, EXPLAIN ANALYZE, Prisma/Drizzle
error-handler	"Add observability"	OpenTelemetry, retry with jitter, circuit breaker, error budgets

Frontend (5)

SKILL	TRIGGER	DEPTH
component-forge	"Create a button"	React/Vue/Svelte, TypeScript strict, WCAG 2.2, compound components
responsive-engine	"Make responsive"	Container Queries, clamp(), :has(), subgrid, touch targets
motion-craft	"Add animation"	WAAPI, Scroll-Driven Animations, FLIP, spring physics, reduced motion
landing-craft	"Design landing page"	CRO frameworks, LIFT Model, A/B testing, Core Web Vitals
ui-ux-pro-max	"Pick a palette"	UI patterns database, color palettes, font pairings, 13 stacks

Mobile (2)

SKILL	TRIGGER	DEPTH
flutter	"Build Flutter app"	Riverpod, Impeller, Dart 3.7+, Pigeon, Clean Architecture (405 lines)
react-native	"Build RN app"	Expo Router, FlashList, Reanimated 4, New Architecture, Hermes (356 lines)

Quality (2)

SKILL	TRIGGER	DEPTH
test-commander	"Add tests"	Testing Trophy, Vitest, MSW, Playwright E2E, visual regression
performance-profiler	"Audit perf"	Lighthouse, Core Web Vitals, bundle analysis, caching strategies

Content & Business (7)

SKILL	TRIGGER	DEPTH
communication	"Write an email"	SBI/BID feedback, tone matrix, cross-cultural, meeting notes (327 lines)
content-marketing	"Write a blog post"	AIDA/PAS/BAB, headline formulas, Cialdini, GEO 2026 (315 lines)
business-proposals	"Create proposal"	Cold email sequences, GBB pricing, SOWs, 10-slide pitch deck
seo-geo	"Optimize search"	SEO+GEO, llms.txt, schema.org, AI Overviews, citability scoring
translate-craft	"Translate to JP"	8 languages, cultural adaptation, ICU syntax, RTL layout
documentation	"Write docs"	READMEs, API docs from OpenAPI, changelogs, KB articles
kami	"Create a PDF"	Professional PDFs, parchment design, 8 templates (409 lines)

Productivity (12)

SKILL	TRIGGER	DEPTH
git-workflow	"Create branch"	Branch naming, atomic commits, conventional commits, PR body gen
windows-powershell	"Check health"	System info, disk cleanup, dev tools install, PS profile
strategy	"Decide approach"	Brainstorming, RICE/ICE scoring, first principles, pre-mortem
design	"Design system"	Brand guidelines, design tokens, visual briefs, SCAMPER
finance	"Plan finances"	5-pillar framework, tax optimization, FIRE, 50/30/20
legal-counsel	"GDPR requirements"	GDPR, AI Act, DSA/DMA, CCPA, HIPAA, DMCA, contract review
whendone-plus	"Notify when done"	Desktop notifications for long-running commands
memory	"Search memory"	ChromaDB persistent memory management
chromadb	"Show stats"	View, search, delete, backup, reset vector DB
cleanup-surgeon	"Clean up files"	File declutter, scan Desktop/Downloads/TEMP, Recycle Bin
portfolio-auto	"Sync portfolio"	Auto-sync GitHub repos to portfolio website
readme-shokunin	"Write README"	README structure, tone by project type, hook-demo-features-why flow

Skill Quality Metrics

After the v4.2 quality audit, all 38 skills meet the minimum standard of workflow, error handling, anti-patterns, and sources. The average skill depth is 170 lines. The deepest are kami (409 lines) and flutter (405 lines). A validation script in CI checks every skill on every commit for required sections.

Memory System v4.2

Three independent layers ensure no context is lost between sessions. ChromaDB provides semantic search; a parallel checkpoint timer saves every 5 minutes; the platform-specific wrapper captures terminal output on exit. Each layer functions independently, providing redundancy.

Layer 1: Agent-Driven Saves

Mechanism: The agent saves context during the session using `chroma-helper.py` via the Bash tool. No MCP server required.

Triggers: Every 3-5 interactions (checkpoint), after decisions (type: decision), after file changes (type: file), after commands (type: command), at session end (type: session_end).

Command:

```
python ~/.shokunin/scripts/chroma-helper.py save "text" "session_id" "type" "tags" "project"
```

Example:

```
python ~/.shokunin/scripts/chroma-helper.py save \
  "Decision: Use RS256 asymmetric signing for JWTs" \
  "session-20260514-103000" "decision" "auth,jwt,security" "myapp"
```

Layer 2: Parallel Checkpoint Timer

Mechanism: Every 5 minutes, a background process saves a timestamped checkpoint to ChromaDB. On Windows, this is a .NET System.Timers.Timer thread. On Linux, it is a bash `sleep 300` loop in a background subshell.

Reliability: This layer is active even if the agent ignores all save instructions. If the session crashes, the checkpoints provide a timeline of activity.

Layer 3: Terminal Capture on Exit

Windows: The wrapper expands the console buffer to 9999 lines, runs opencode, then reads the buffer via `$host.UI.RawUI.GetBufferContents`. Maximum capture: 5000 lines (performance ceiling).

Linux: The wrapper wraps opencode with the `script` command, which captures all terminal I/O in real-time. No line limit. This is objectively superior to the Windows approach.

Post-processing: ANSI escape sequences are stripped. The output is saved as both a raw log (.log) and a parsed markdown file (.md) with extracted decisions and commands.

Fallback Chain

```
1. chroma-helper.py save via Bash   (primary, always available)
2. store_context via MCP server     (secondary, if MCP is running)
3. sessions/*.md markdown file      (tertiary, filesystem fallback)
4. current-session.json metadata    (last resort, session identity only)
```

Data Format

```
{
  "type": "decision | file | command | checkpoint | session_end",
  "timestamp": "2026-05-14T10:30:00Z",
  "project": "my-project",
  "tags": ["auth", "jwt", "decision"],
  "session_id": "session-20260514-103000-A7F3",
  "text": "Descriptive context: what was done, why, and the result"
}
```

Storage

LOCATION	FORMAT	SIZE
ChromaDB vector database	sqlite3 + embeddings	~400 KB base + ~1 KB per entry
Session raw logs	Plain text (.log)	1-10 KB per session
Session summaries	Markdown (.md)	0.5-3 KB per session
Active session metadata	JSON	~200 bytes
MCP server log	Plain text	~50 bytes per operation

MCP Servers

MCP (Model Context Protocol) servers give the AI agent direct system access. They start on demand via opencode.json, consume no resources when idle, and work identically on Windows and Linux.

SERVER	TOOLS	STARTUP	CONFIG
filesystem	read_file, write_file, edit_file, search_files, list_directory	`npx @modelcontextprotocol/server-filesystem`	Path validated root directory
fetch	fetch, fetch_json	`npx @modelcontextprotocol/server-fetch`	No config needed
memory	store_context, search_context, get_session_summary	`python mcp-server.py`	317 lines, JSON-RPC 2.0, logging built-in

Memory MCP Server

The memory server is 317 lines of Python implementing JSON-RPC 2.0 over stdin/stdout. It supports three tools and logs all operations to `mcp-server.log`.

```
// List available tools
echo '{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{}}' | python mcp-server.py

// Store a context entry
echo '{"jsonrpc":"2.0","id":2,"method":"tools/call",
  "params":{"name":"store_context",
    "arguments":{"text":"Auth uses RS256","session_id":"s-123",
      "type":"decision","tags":["auth"]}}}' | python mcp-server.py

// Search stored context
echo '{"jsonrpc":"2.0","id":3,"method":"tools/call",
  "params":{"name":"search_context",
    "arguments":{"query":"RS256 authentication","n_results":5}}}' | python mcp-server.py
```

The MCP server has automatic type validation, integrated markdown fallback writing, and comprehensive error logging. All operations are validated before being passed to ChromaDB.

Subagents

Four specialized subagents operate in parallel with their own model backends. They handle code review and debugging without blocking the main conversation thread.

SUBAGENT	PRIMARY MODEL	OFFLINE FALLBACK	PURPOSE
code-review	NVIDIA Kimi K2.6	Ollama Qwen3 14B	Analyze diffs for bugs, security issues, code smells, anti-patterns
code-review-local	Ollama Qwen3 14B	-	Offline-first code review with no internet dependency
debugger	NVIDIA Kimi K2.6	Ollama Qwen3 14B	Root cause analysis from stack traces and error logs
debugger-local	Ollama Qwen3 14B	-	Offline debugging without internet access

Subagents are configured in `opencode.json` and use the model key from the providers section. The offline fallback ensures the ecosystem works without internet access, using locally running Ollama models.

08 · CROSS-PLATFORM

Cross-Platform Architecture

The same 38 skills, ChromaDB memory, MCP servers, and subagents work identically on Windows and Linux. Only the wrapper scripts, scheduling, and profile configuration are platform-specific.

Shared (identical across platforms)

COMPONENT	TYPE
38 SKILL.md files	Markdown
chroma-helper.py	Python (cross-platform paths via platform.system())
mcp-server.py	Python (JSON-RPC 2.0)
CLAUDE.md + AGENTS.md	Markdown (runtime instructions)
opencode.json	JSON (MCP + provider config)
4 subagents	Config-based
read-transcript.py	Python (cross-platform ANSI stripping)

Platform-Specific

CAPABILITY	WINDOWS	LINUX ADVANTAGE
Terminal capture	Buffer API via $host.UI.RawUI (5000 lines, PS5.1)	script command (unlimited, no line limit)
Session wrapper	run-opencode.ps1 (192 lines)	run-opencode.sh (105 lines)
Installer	install.ps1 (467 lines, refactored)	install.sh (206 lines, clean)
Profile shadow	PowerShell function in $PROFILE	bash/zsh function in .bashrc/.zshrc
Task scheduling	Task Scheduler (weekly)	crontab (weekly)
Healthcheck	memory-healthcheck.ps1 (8 tests)	memory-healthcheck.sh (8 tests)
File cleanup	scan-cleanup.ps1 (Microsoft.VisualBasic Recycle Bin)	scan-cleanup.sh (trash directory)
Test suite	test-memory.ps1 (12 automated checks)	test-memory.sh (10 checks)

Cross-platform path resolution is handled by chroma-helper.py using `os.getenv("USERPROFILE") or os.getenv("HOME") or os.path.expanduser("~")`. This ensures the same script works on both operating systems without modification.

09 · MULTI-RUNTIME

Multi-Runtime Support

The ecosystem is not tied to any single AI coding runtime. Skills, memory, MCP servers, and scripts work across six different runtimes through configuration templates and runtime-specific instruction files.

RUNTIME	SKILLS	MEMORY	MCP	CONFIG LOCATION
OpenCode	Native	Native	opencode.json	Built-in
Claude Code	Reads SKILL.md	Via MCP + chroma-helper	claude-code.json (template)	.pack/templates/claude-code.json
Cline	Reads SKILL.md	Via .clinerules	cline-settings.json (template)	.pack/templates/cline-settings.json
Cursor	Reads SKILL.md	Via .cursor/rules/*.mdc	cursor-mcp.json (template)	.pack/templates/cursor-mcp.json
Continue.dev	Reads SKILL.md	Via .continue/rules/*.md	continue-config.yaml (template)	.pack/templates/continue-config.yaml
Windsurf	Reads SKILL.md	Via .windsurf/rules/*.md	Via UI (no config file)	.pack/rules/windsurf-memory.md

All template configurations are in `.pack/templates/` (4 files) and runtime rules in `.pack/rules/` (4 files). No existing files are modified to add a new runtime. The memory instructions in CLAUDE.md are also available as `CLAUDE.md.template` with generic placeholders (`{{USERNAME}}` , `{{LANGUAGE}}`) for users who want to customize.

> Vendor independence: If OpenCode changes its API or is discontinued, the ecosystem migrates to any of the other five supported runtimes by copying a configuration template. Skills, memory, and scripts remain unchanged.

10 · SCRIPTS

Scripts & Automation

Twenty scripts form the operational backbone of the ecosystem. They handle session management, memory operations, health checks, file cleanup, code validation, and weekly maintenance.

Core Python (cross-platform)

SCRIPT	LINES	PURPOSE
chroma-helper.py	86	Direct ChromaDB access via save, search, count, recent commands. Cross-platform paths. Used by all other scripts.
mcp-server.py	317	JSON-RPC 2.0 MCP server for memory operations. Three tools with logging and error handling.
read-transcript.py	65	Cross-platform transcript parser: strips ANSI, extracts decisions/commands, outputs structured markdown.

Windows PowerShell (8 scripts)

SCRIPT	LINES	PURPOSE
run-opencode.ps1	192	Main session wrapper: env vars, MCP healthcheck, buffer capture, ChromaDB save, checkpoint timer, signal handling
search-memory.ps1	55	Unified search: ChromaDB semantic + recent entries fallback. Never returns empty.
save-memory.ps1	57	Save with 2 retries via chroma-helper.py, markdown fallback if all retries fail
memory-healthcheck.ps1	131	8 tests: Python, chromadb, chroma-helper save/search, MCP tools, script syntax, config, storage
test-memory.ps1	127	12 automated tests for comprehensive memory system validation
scan-cleanup.ps1	171	Desktop/Downloads/TEMP scanner, AI classification, Recycle Bin via Microsoft.VisualBasic
seed-memory.ps1	30	Import session markdown files into ChromaDB for historical context
validate-skills.ps1	58	Validate all 38 skills for required sections: frontmatter, workflow, error handling, sources

Linux bash (6 scripts)

SCRIPT	LINES	PURPOSE
run-opencode.sh	105	Linux wrapper: script capture, checkpoint timer, ChromaDB save, signal cleanup
memory-healthcheck.sh	53	Linux validation: Python, chromadb, save/search, MCP, configs, storage
test-memory.sh	60	Linux test suite: 5 sections, 10 automated checks
scan-cleanup.sh	55	Linux cleanup: TEMP files, shokunin PDFs, Desktop files
weekly-maintenance.sh	26	Crontab-ready: ChromaDB backup, log cleanup, disk space report
profile.sh	41	bash/zsh aliases for git, npm, docker, mkcd, and opencode wrapper function

Additional (3)

FILE	LINES	PURPOSE
profile.ps1	66	PowerShell profile extracted from install.ps1: git/npm/docker functions, PSReadLine config, opencode shadow
weekly-healthcheck.ps1	59	Task Scheduler healthcheck: disk space, memory backup, log cleanup
read-transcript.ps1	72	Windows transcript parser (legacy, superseded by read-transcript.py)

Quality & CI/CD

Three GitHub Actions workflows validate every component on every push. The CI pipeline tests Python, PowerShell, bash, and configuration files across Ubuntu and Windows runners.

CI Workflows

WORKFLOW	JOBS	PLATFORM	VALIDATES
ci.yml	validate	Ubuntu	38 skills frontmatter + workflow + error handling + sources; JSON config; bash + Python syntax; template integrity
memory-tests.yml	test-python, test-scripts, test-linux, validate-config	Ubuntu + Windows	ChromaDB import, MCP protocol, PS1 parsing, bash syntax, cross-platform paths, JSON validity
release.yml	release (on tag push v*)	Ubuntu	Generates changelog, builds archive, creates GitHub Release

Quality Standards

- Senior Engineer Coding: All scripts use guard clauses, named constants, no magic numbers, edge case handling for null/empty/error states
- Error Handler: Retry with jitter (2 attempts with exponential backoff), structured fallback chain with logging, never silent catch
- OWASP security: No secrets in code, validated input (sed delimiter guard for API keys), principle of least privilege, no stack traces in user output
- Idempotent installer: Backup before write, skip existing entries, safe to re-run without side effects
- Cero regresiones: All changes are additive. Existing content is never modified, only extended

Audit Results: 10 Critical Bugs Found and Fixed

During a comprehensive code audit, ten critical bugs were identified and resolved. All fixes were surgical (1-3 lines each) and did not affect any existing functionality.

#	FILE	BUG	IMPACT	FIX
1	install.ps1	Set-Alias with arguments broken (PowerShell limitation)	All git/npm/docker aliases silently failed	Replaced with function definitions `function ga { git add -A }`
2	run-opencode.ps1	Fallback save wrote filename as file content	Session fallback saved garbage data	Changed to write `$summaryText` variable
3	weekly-healthcheck.ps1	Disk percentage formula: `Used/Used*100/(Used/Used) = 100 always`	Healthcheck always reported 100% disk usage	Fixed to `Used/(Used+Free)*100`
4	scan-cleanup.ps1	Shell.Namespace(0) targets Desktop, not Recycle Bin	Deleted files appeared on Desktop instead of Recycle Bin	Changed to Microsoft.VisualBasic.DeleteFile with SendToRecycleBin
5	memory-healthcheck.ps1	Test overwrites active current-session.json	Disrupts active session tracking	Removed destructive write
6	mcp-server.py	Variable shadowing: `entry_type` from request overwritten in loop	search_context type filter silently ignored	Renamed to `filter_type` and `meta_type`
7	install.ps1	`npx --version` only prints version, does not install	MCP servers never installed by installer	Changed to `npm install -g`
8	profile.sh	Redefines `touch()` which truncates files instead of updating timestamps	System scripts using touch could destroy files	Removed dangerous function override
9	memory-healthcheck.sh	Single quotes prevent variable expansion: `grep -q '$TEST_ID'`	Search test always fails (searches for literal `$TEST_ID`)	Changed to double quotes
10	run-opencode.ps1	Hardcoded path to opencode binary	Fails if npm prefix is not default	Changed to dynamic `Get-Command opencode`

Getting Started

Installation is a single command on both Windows and Linux. The installers are idempotent, create backups of existing configurations, and verify prerequisites before making any changes.

Installation

Windows (PowerShell 5.1+):

```
irm https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.ps1 | iex
```

Linux (bash 4+):

```
bash <(curl -sL https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.sh)
```

Prerequisites

SOFTWARE	MINIMUM VERSION	INSTALL COMMAND (WINDOWS)	INSTALL COMMAND (LINUX)
PowerShell / bash	5.1+ / 4+	Built into Windows 10/11	Built into all distributions
Node.js	18+	`winget install OpenJS.NodeJS.LTS`	`apt install nodejs` or `nvm`
Python	3.11+	`winget install Python.Python.3.12`	`apt install python3 python3-pip`
Git	Any	`winget install Git.Git`	`apt install git`
OpenCode	Any	`npm install -g opencode`	`npm install -g opencode`

First Session

After installation, open a new terminal and run:

```
opencode
```

The profile wrapper function routes the command through the session wrapper, which sets environment variables, checks MCP server health, and launches the AI coding runtime. The agent reads CLAUDE.md, searches memory for past context, and presents any relevant findings.

Daily Commands

ACTION	WINDOWS	LINUX
Start session	`opencode`	`opencode`
Test memory	`.\test-memory.ps1`	`./test-memory.sh`
Check health	`.\memory-healthcheck.ps1`	`./memory-healthcheck.sh`
Search memory	`search-memory.ps1 -Query "topic"`	`python chroma-helper.py search "topic"`
Clean up files	`.\scan-cleanup.ps1 -Scan`	`./scan-cleanup.sh`
Git status	`gst`	`gst`
Git commit	`gc "message"`	`gc "message"`
npm install	`ni`	`ni`

Workflow Examples

The following examples demonstrate how the ecosystem operates in practice. Each example shows the skills activated, the agent actions, and the memory entries created.

Authentication Implementation

User: “Add Google OAuth login to my Express app.”

1. auth-architect activates: generates OAuth2 + PKCE flow, JWT with RS256 rotation, session management with rate limiting per IP (5 req/min).
2. db-sculptor activates: creates User schema with Prisma, migration script, indexes on `email` and `provider` columns.
3. error-handler activates: wraps auth routes with structured error handling, logs failed attempts to OpenTelemetry.
4. test-commander activates: generates integration tests with mocked OAuth provider, covering success and error flows.
5. Memory: saves “Decision: RS256 asymmetric signing for JWTs” (type: decision), “File: prisma/schema.prisma modified” (type: file).

Production Deployment

User: “Dockerize and deploy to Kubernetes.”

1. docker activates: multi-stage Dockerfile (builder pattern), BuildKit cache mounts, .dockerignore, docker-compose for local dev with hot reload.
2. kubernetes activates: Deployment with rolling update, Service (ClusterIP), Ingress with TLS, HPA (CPU > 70%), PDB (min 2 available).
3. ci-cd activates: GitHub Actions pipeline with lint, test, build, scan, deploy, healthcheck, and rollback steps.
4. db-admin activates: PostgreSQL backup strategy with WAL archiving, PgBouncer connection pooling config.

Incident Response

User: “API is returning 502 errors, investigate.”

1. runbook-gen activates: severity assessment (SEV2), war room channel, escalation path, customer communication template.
2. error-handler activates: analyzes recent logs for 502 patterns, identifies upstream timeout as root cause.
3. debugger subagent: deep analysis of stack traces, confirms connection pool exhaustion.
4. memory: searches past sessions for similar incidents, finds previous fix: “Increased PgBouncer pool from 10 to 25” (type: checkpoint).

System Declutter

User: “Clean up old files I do not need.”

1. cleanup-surgeon activates: runs scan-cleanup.ps1 (or .sh on Linux).
2. Scans Desktop, Downloads, TEMP for files older than thresholds.
3. AI classifies each file: safe (green, auto-delete), review (yellow, ask user), keep (red, skip).
4. Presents summary: “19 files, 85 MB. 12 safe to delete, 5 review, 2 keep.”
5. On confirmation, moves files to Recycle Bin via Microsoft.VisualBasic (Windows) or trash directory (Linux).
6. Memory: “Cleanup: 8 files recycled, 62 MB freed.” (type: session_end).

Content Production

User: “Write a launch announcement blog post and export as PDF.”

1. content-marketing activates: AIDA framework, PAS headline, SEO keyword research, Cialdini principles applied.
2. seo-geo activates: structured data markup, llms.txt entry, AI Overview optimization.
3. kami activates: parchment design system, generates HTML post, converts to professional PDF.
4. Memory: “Blog post created: launch announcement.” (type: file).

Technical Specifications

Repository Statistics

METRIC	VALUE
Total commits	52
Skills	38 (covering 9 domains)
Scripts total	20
PowerShell scripts	8
bash scripts	6
Python scripts	3
CI workflows	3
Documentation PDFs	2 (Ultimate Guide + Quickstart)
Installer lines (Windows)	467
Installer lines (Linux)	206
Template configs	4 (for 5 additional runtimes)
Runtime rule files	4

Memory System Specifications

PARAMETER	VALUE
Vector database	ChromaDB (PersistentClient, sqlite3 backend)
Database size (base)	~400 KB
Size per entry	~1 KB
Current entries	11+
Search method	Semantic embedding (ChromaDB query)
Fallback method	Recent entries via collection.get()
Checkpoint interval	300 seconds (5 minutes)
Console buffer (Windows)	9999 lines expanded, 5000 read
Terminal capture (Linux)	Unlimited (script command)
Save retries	2 attempts before fallback
MCP server lines	317 (Python, JSON-RPC 2.0)
chroma-helper.py lines	86 (direct ChromaDB access)

Version History

VERSION	DATE	KEY CHANGES
v1.0	2025 Q4	Initial 29 skills
v2.0	2026 Q1	All skills rewritten with workflows, checklists, anti-patterns, sources
v3.0	2026.04	12 skills upgraded to v3.0, memory MCP server, subagents, superpowers plugin, Telegram bot
v4.0	2026.05.14	Memory rewrite: structured data, run-opencode.ps1, chroma-helper.py, 10 critical bugs fixed
v4.1	2026.05.14	Checkpoint timer, CI pipeline, profile shadow, 3 capture strategies, search-memory fix
v4.2	2026.05.14	Linux port, multi-runtime support, 13 skills upgraded, 10/10 quality push, release workflow

15 · ROADMAP

Roadmap

v5.0 (2026 Q3)

- Cross-platform tests: Full CI test suite running on both Windows and Linux runners for all 20 scripts
- Skill benchmarks: Comparative analysis of task completion with and without skills vs general prompts
- Memory dashboard: Web-based UI for browsing, searching, and managing memory entries
- Community contribution guide: Structured process for external skill contributions with automated review

v5.1 (2026 Q4)

- Memory export/import: Share memory contexts between machines and team members
- Skill marketplace: Curated index of community-contributed skills with quality scoring
- Performance profiler integration: Automated session analytics (memory usage, query latency, save frequency)

v6.0 (2027)

- macOS support: Port scripts to zsh profile, launchd scheduling, osascript notifications
- Collaborative memory: Shared ChromaDB instances for team context
- IDE plugins: Native VS Code and JetBrains extensions for Shokunin integration

> Governance: Shokunin is and will remain MIT-licensed open source. The roadmap is guided by real user feedback, not venture capital.

APPENDIX A

Quick Reference

Key Paths

COMPONENT	PATH
38 Skills	~/.config/opencode/skills/
OpenCode config	~/.config/opencode/opencode.json
CLAUDE.md (local)	~/.claude/CLAUDE.md
AGENTS.md (local)	~/AGENTS.md
CLAUDE.md template	.pack/CLAUDE.md.template
ChromaDB data	~/.shokunin/memory/chroma_db/
Session logs	~/.shokunin/memory/sessions/
Current session info	~/.shokunin/current-session.json
MCP server log	~/.shokunin/memory/mcp-server.log
Cleanup log	~/.shokunin/logs/cleanup.log
Weekly backups	~/.shokunin/backups/
MCP templates	.pack/templates/ (4 files)
Runtime rules	.pack/rules/ (4 files)
GitHub repository	github.com/EliasOulkadi/shokunin

Environment Variables

VARIABLE	SET BY	PURPOSE
`SHOKUNIN_SESSION_ID`	run-opencode wrapper	Current session UUID (e.g. session-20260514-103000-A7F3)
`SHOKUNIN_PROJECT`	run-opencode wrapper	Working directory at session start
`SHOKUNIN_MCP_HEALTHY`	run-opencode wrapper	"1" if MCP server responded, "0" otherwise
`SHOKUNIN_LAST_SESSION`	run-opencode wrapper	Previous session ID (set on exit)

Memory Commands

ACTION	COMMAND (WINDOWS)	COMMAND (LINUX)
Search by topic	`search-memory.ps1 -Query "auth jwt"`	`python chroma-helper.py search "auth jwt"`
List recent	`search-memory.ps1 -Query ""`	`python chroma-helper.py recent 5`
Save a note	`save-memory.ps1 -Text "..." -Tags decision`	`python chroma-helper.py save "..." ...`
Count entries	`python chroma-helper.py count`	`python chroma-helper.py count`
Check health	`.\memory-healthcheck.ps1`	`./memory-healthcheck.sh`
Run all tests	`.\test-memory.ps1`	`./test-memory.sh`
Import sessions	`.\seed-memory.ps1`	Manual via chroma-helper.py
Validate skills	`.\validate-skills.ps1`	Manual
Clean files	`.\scan-cleanup.ps1 -Scan`	`./scan-cleanup.sh`

Shell Aliases

ALIAS	COMMAND
`gst`	git status
`ga`	git add -A
`gc`	git commit -m (Windows: `gc "message"` , Linux: `gc "message"`)
`gp`	git push
`gl`	git pull --ff-only
`ni`	npm install
`nrd`	npm run dev
`nrb`	npm run build
`nt`	npm test
`dps`	docker ps
`dlog`	docker logs -f
`dstop`	docker stop
`ll`	ls -la (Linux) / Get-ChildItem (Windows)
`mkcd`	mkdir + cd

> **Links**
>
> GitHub repository: `github.com/EliasOulkadi/shokunin`
>
> Install Windows: `irm https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.ps1 | iex`
>
> Install Linux: `bash <(curl -sL https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.sh)`
>
> PDF Guides: `github.com/EliasOulkadi/shokunin/blob/master/docs/`
>
> License: MIT — Created by Elias Oulkadi — Version 4.2 — May 2026